

Informe de pruebas de la pasarela de pagos

Integración con Wompi · Verificación funcional y de seguridad

COMERCIO	AMBIENTE	PASARELA	FECHA
Natalia Andrea Sánchez Pérez	Pruebas (sandbox)	WOMPI	7 de octubre de 2026, 10:56 a. m.

Resumen

44 PASAN	0 FALLAN	0 SIN APLICAR
-------------	-------------	------------------

Se ejecutaron **44 pruebas** sobre la integración de pagos: credenciales, firma de integridad del checkout, verificación de los avisos de la pasarela, intentos de abuso, máquina de estados, reconciliación, superficie expuesta y límites de uso.

Veredicto: la pasarela funciona de punta a punta. Se completaron pagos reales en el sandbox de Wompi, con tarjeta aprobada y con tarjeta rechazada, conduciendo el checkout real en un navegador. En los dos casos llegó el aviso de Wompi, **nuestra verificación del checksum lo aceptó**, el pedido cambió de estado y el inventario se movió —o no se movió— como correspondía.

Eso cierra la única duda de fondo que quedaba: el algoritmo del checksum que implementamos **es** el de Wompi, pese a que el ejemplo resuelto de su documentación no reproduzca su propio hash (H-4).

La integración además resiste los ataques que se le probaron: no se puede cambiar el monto de un pedido, no se puede falsificar un aviso, un aviso repetido no duplica nada y dos compras simultáneas no venden la misma unidad. Durante las pruebas aparecieron **cuatro defectos reales, los cuatro corregidos**, y queda **un riesgo abierto** que no es de configuración sino de diseño (R-1).

Pagos reales completados en sandbox

REFERENCIA	TARJETA	WOMPI	PEDIDO	INVENTARIO
AU-11682-b7c7	4242...4242	APPROVED	PAID	SALE -1 · stock 20 → 19
AU-11683-93c1	4111...1111	DECLINED	CANCELLED	sin movimiento · reserva liberada

En los dos casos el aviso de Wompi quedó registrado con `checksumOk = true` y `aplicado = true`.

Qué es real en estas pruebas

Un informe de seguridad vale lo que valga su metodología, así que conviene ser explícito sobre dónde termina lo comprobado:

- **El API de Wompi es real.** Las credenciales se validan contra `sandbox.wompi.co` ; no hay simulación de la pasarela en ninguna prueba.
- **Nuestra API es real.** Las pruebas entran por HTTP igual que lo haría el navegador, contra el servidor corriendo, y comprueban el efecto en la base de datos.
- **Los avisos de las pruebas de ataque se fabrican aquí** y se firman con el secreto de eventos real, porque hay que poder manipularlos a voluntad. Pero **el circuito se cerró con avisos emitidos por Wompi**: E2E-01 comprueba la bitácora de lo que llegó de verdad al webhook.
- **Los pagos se hicieron conduciendo el checkout real** en un navegador: elegir medio de pago, teclear la tarjeta, aceptar los términos y enviar.
- **Las pruebas limpian lo que ensucian.** Las reservas que crean se liberan al terminar por la vía normal del servicio, sin escribir el stock a mano.

El banco de pruebas es reproducible con `npm run payments:test` y devuelve código de salida distinto de cero si algo falla, así que sirve tal cual en una tubería de integración continua.

Arquitectura de seguridad

Nueve capas, cada una contra una forma concreta de perder dinero o mercancía. Ninguna depende de que las demás funcionen.

Firma de integridad

Al abrir el checkout

Impide: Que nadie cambie el monto del pedido en el navegador antes de pagar.

SHA-256 sobre referencia + monto + moneda + expiración + secreto de integridad. Wompi recalcula la firma y rechaza la transacción si el monto no es el firmado. El secreto nunca sale del servidor.

FIRM-01 a FIRM-05

Checksum del aviso

Al recibir un webhook

Impide: Que cualquiera pueda mandarnos «pagado» y llevarse la mercancía.

SHA-256 sobre las propiedades firmadas + timestamp + secreto de eventos, comparado en tiempo constante para que no se pueda adivinar byte a byte midiendo tiempos de respuesta.

AVIS-01 a AVIS-06, ATAQ-01

Verificación del monto

Al aplicar un aviso

Impide: Que una transacción legítima de \$1.000 confirme un pedido de \$300.000.

Antes de aprobar nada se compara el monto que dice la pasarela con el que nosotros cobramos. Si no coinciden, el pago no se aprueba y la discrepancia queda registrada.

ATAQ-02

Ventana de frescura

Al recibir un webhook

Impide: Que un aviso legítimo capturado hace días se reenvíe como si fuera nuevo.

Se descarta todo aviso de más de 48 h, con 5 minutos de tolerancia hacia el futuro por desfase de relojes. La ventana cubre con margen los tres reintentos de Wompi.

AVIS-07 a AVIS-09

Idempotencia

Al aplicar un aviso

Impide: Que los reintentos de Wompi descuenten el stock dos veces o cobren por partida doble.

Cada aviso lleva una huella y la pareja (pago, huella) es única en la base. Un reenvío se reconoce, se responde 200 para que Wompi deje de insistir, y no se vuelve a aplicar.

ATAQ-03, ATAQ-04

Guardia de transiciones

Al cambiar de estado

Impide: Que un aviso desordenado cancele un pedido ya cobrado.

Los avisos llegan fuera de orden. Un pago aprobado no admite volver a pendiente ni pasar a rechazado; en cambio un pago caducado sí admite aprobación tardía, porque si el dinero entró, entró.

EST-01 a EST-03

Reserva atómica de stock

Al crear el intento

Impide: Que dos compras simultáneas vendan la misma unidad.

La condición de stock va DENTRO del UPDATE, no en una lectura previa. La base decide quién gana; el perdedor recibe 409. El stock nunca se escribe a mano: todo cambio es un movimiento de inventario dentro de una transacción.

ATAQ-06, ATAQ-08

Superficie mínima

En todo el perímetro

Impide: Que haya más puerta de la que hace falta.

La ruta de pago simulado no se registra con Wompi activo. La consulta pública de un pago no devuelve datos personales. Las referencias se validan por forma antes de tocar la base. Al exponer el webhook a internet solo se publica esa ruta, no la API.

ATAQ-05, SUP-01 a SUP-04

Límite de peticiones

En las rutas públicas

Impide: Que una ráfaga aparte todo el inventario sin pagar nada.

20 intentos por minuto y 60 consultas por minuto, por IP. El webhook va con un tope muy alto (300/min) a propósito: un 429 gastaría uno de los tres reintentos de Wompi y eso perdería pagos de verdad.

ATAQ-10, ATAQ-11

Hallazgos

Lo que apareció al probar. Los tres primeros eran defectos reales y están corregidos; los dos últimos son constancia de cosas que conviene no olvidar.

H-0 **CORREGIDO** Con APP_URL en localhost el checkout no abre, en absoluto

QUÉ PASABA

Wompi rechaza con un 403 de CloudFront («Request blocked») cualquier redirect-url que apunte a localhost o 127.0.0.1, con http y con https. La configuración por defecto del proyecto era APP_URL=http://localhost:5173.

EVIDENCIA

Aislado parámetro a parámetro contra el checkout real: con los otros doce parámetros la URL responde 200, y basta añadir el redirect-url a localhost para que devuelva 403. Un dominio cualquiera pasa, incluso por http simple. Esta es la razón de que la integración nunca se hubiera completado de punta a punta.

QUÉ SE HIZO

El arranque lo detecta y avisa a gritos; en producción aborta. Para probar en local, APP_URL apunta al mismo túnel con el que se reciben los avisos. La prueba FIRM-06 no se conforma con mirar la forma de la URL: pide el checkout de verdad y exige un 200.

El síntoma no se parece a la causa: la clienta ve una página de error de Amazon sin una sola pista, y en nuestro servidor no queda ni rastro.

H-1 **CORREGIDO** Las reservas de stock no tenían ningún freno

QUÉ PASABA

Crear un intento de pago aparta unidades durante 15 minutos y no exige cuenta, pago ni nada. No había límite de peticiones.

EVIDENCIA

Medido contra esta misma tienda: 10 reservas aceptadas en 355 ms desde una sola IP, dejando el producto sin unidades disponibles durante un cuarto de hora. Repetido sobre el catálogo, apaga la tienda sin que entre un peso.

QUÉ SE HIZO

Se añadió límite de peticiones por IP en las rutas públicas de pago (20/min en intentos, 60/min en consultas, 300/min en el webhook), más un tope de 256 KB de cuerpo y una decisión explícita sobre si crear la cabecera X-Forwarded-For.

MITIGADO, NO CERRADO. Ver R-1 en riesgos abiertos.

H-2 **CORREGIDO** La ventana de frescura rechazaba el último reintento de Wompi

QUÉ PASABA

Los avisos de más de 24 h se descartaban por viejos. Wompi reintenta un aviso no confirmado a los 30 minutos, a las 3 horas y a las 24 horas.

EVIDENCIA

El tercer reintento sale exactamente a las 24 h del original, así que caía fuera de la ventana por segundos y lo rechazábamos nosotros mismos. Es justo el que más falta hace: el que llega después de dos fallos.

QUÉ SE HIZO

Ventana ampliada a 48 h. No debilita la defensa: un aviso repetido ya rebota antes por la unicidad de (pago, huella), y la ventana solo sirve para descartar capturas viejas.

Prueba de regresión permanente: AVIS-08.

H-3 **CORREGIDO** No se podía reconciliar un cobro por referencia

QUÉ PASABA

El adaptador solo sabía consultar por id de transacción, que llega en el webhook o en la URL de retorno. Si se perdían los dos, el cobro se quedaba pendiente para siempre con el stock bloqueado.

EVIDENCIA

Es el caso real de quien paga y cierra el navegador, o se queda sin batería. Se comprobó contra el API de Wompi que GET /v1/transactions?reference= sí responde, pero solo autenticado con la llave privada, que no estaba configurada.

QUÉ SE HIZO

Se añadió WOMPI_PRIVATE_KEY y se implementó la búsqueda por referencia. Las consultas al API pasan a usar la llave privada, que además es lo que Wompi recomienda desde que retiró el soporte a consultar transacciones desde el frontend.

Si una referencia tiene varios intentos, manda el aprobado; si no, el más reciente. Prueba REC-01.

H-4 **VERIFICADO**

El ejemplo resuelto de la documentación de Wompi no reproduce su propio checksum

QUÉ PASABA

La documentación publica una cadena de ejemplo, un secreto de ejemplo y el checksum resultante. Hasheando la cadena que publican con el secreto que publican sale otro hash.

EVIDENCIA

Comprobado en esta sesión. Esperado 3476DDA5...8BD0; calculado 5A18EC5E...EFBE. La explicación más probable es que anonimizaran el secreto pero dejaran el hash real.

QUÉ SE HIZO

Ninguno posible por nuestra parte. Se implementa la regla en prosa, que sí es coherente entre la documentación en español, la inglesa y las implementaciones de terceros.

Por esto existe E2E-01: solo un aviso emitido por Wompi demuestra que nuestro algoritmo es el suyo.

H-5

CORREGIDO

La primera versión de la prueba E2E daba falsas alarmas

QUÉ PASABA

E2E-01 contaba como «aviso real de Wompi» cualquier petición que pasara por el proxy del túnel.

EVIDENCIA

Las sondas con las que se verifica que el túnel funciona (`{"x":1}`) entraban como avisos reales rechazados, y la prueba concluía que el algoritmo del checksum estaba roto. Lo estaba midiendo contra nuestro propio tráfico de verificación.

QUÉ SE HIZO

Ahora solo cuentan las entradas con forma de aviso de Wompi: `event`, `timestamp`, `signature.properties` y `signature.checksum`. Las sondas se reportan aparte.

Una prueba de seguridad que grita sin motivo acaba ignorándose, que es peor que no tenerla.

Riesgos abiertos

Lo que sigue sin resolver, con lo que haría falta para cerrarlo. R-1 es el que de verdad importa.

R-1 **ALTA** El inventario se puede agotar sin pagar

SITUACIÓN

El límite de peticiones frena el barrido del catálogo entero —de segundos a varios minutos— pero no salva a un producto concreto: poner el tope por debajo del stock de un artículo dejaría fuera a clientes reales. Un producto con diez unidades sigue siendo vaciable en menos de un minuto desde una sola IP, y repartido entre muchas IPs el problema escala.

CÓMO SE CIERRA

Apartar stock al abrir el checkout en vez de al crear el intento; acortar la ventana de reserva; topar las reservas pendientes por IP. Las tres son cambios de producto, no de configuración.

R-2 **MEDIA** La caducidad de reservas es oportunista, no programada

SITUACIÓN

Las reservas vencidas se liberan aprovechando que alguien compra. En una tienda sin tráfico, las unidades de un carrito abandonado se quedan bloqueadas indefinidamente.

CÓMO SE CIERRA

Una tarea programada cada minuto, o un worker. Hoy está así para que la demo funcione sin infraestructura extra.

R-3 **MEDIA** La URL de eventos es un túnel temporal

SITUACIÓN

La URL de pruebas vive mientras el túnel esté arriba y cambia en cada reinicio. Para producción hace falta un dominio propio con TLS, y registrar URLs distintas para sandbox y para producción.

CÓMO SE CIERRA

Desplegar la API tras un dominio estable y registrar esa URL en el Dashboard de Wompi, ambiente por ambiente.

R-4 **MEDIA** Sin devoluciones ni anulaciones desde el panel

SITUACIÓN

Un pago aprobado por error solo se puede deshacer desde el Dashboard de Wompi, y esa anulación no vuelve automáticamente al inventario.

CÓMO SE CIERRA

Implementar la anulación contra el API de Wompi y el movimiento de inventario que la acompaña.

R-5 **MEDIA** Las credenciales se compartieron por chat

SITUACIÓN

Las cuatro credenciales de sandbox llegaron por mensaje. Para pruebas el daño posible es acotado —no hay dinero real— pero quedan en un historial de conversación.

CÓMO SE CIERRA

Rotarlas desde el Dashboard cuando termine la integración. Y para las de PRODUCCIÓN: no pasarlas nunca por chat ni por correo; cargarlas directamente en el servidor o en un gestor de secretos.

URL de eventos

Wompi avisa de cada cambio de estado de una transacción contra una URL pública nuestra. Es lo que confirma los pagos: sin ella, un cobro aprobado no llega nunca a convertirse en pedido.

La ruta es:

```
POST <dominio>/api/payments/webhook/wompi
```

Para probar en local se expone con un túnel, pero **no se tuneliza la API entera**: eso publicaría en internet el login del panel con credenciales de demo. Hay un proxy (`npm run webhook:tunnel`) que solo deja pasar esa ruta y ese método, limita el cuerpo a 64 KB y deja bitácora de todo lo que llega. Comprobado desde fuera: el webhook responde y el resto de la API devuelve 404.

1. Arrancar el proxy y el túnel.
2. Pegar la URL resultante en el Dashboard de Wompi, en **Configuración** → **URL de eventos**, del ambiente que corresponda.
3. Pagar una transacción de prueba y volver a correr `npm run payments:test` : E2E-01 deja de estar «sin aplicar».

Sandbox y producción llevan URLs distintas. Para producción hace falta un dominio propio con TLS: la URL del túnel cambia en cada reinicio.

Resultados detallados

A · Credenciales y entorno

ID	PRUEBA	GRAV.	RESULT.
CRED-01	La llave pública es válida en el API de Wompi POR QUÉ Una llave mal copiada no falla al arrancar: falla en el primer cobro real, delante de una clienta. OBSERVADO Comercio «Natalia Andrea Sanchez Perez», activo.	Crítica	Pasa
CRED-02	La llave privada autentica contra el API POR QUÉ Sin ella no se puede reconciliar un cobro por referencia, que es la única salida cuando se pierde el aviso y la clienta no vuelve. OBSERVADO Autenticada: 200 y lista vacía para una referencia inexistente.	Crítica	Pasa
CRED-03	Las cuatro credenciales son del mismo ambiente POR QUÉ Mezclar sandbox y producción hace que el checkout cobre de mentira mientras el servidor consulta la cuenta real: los pedidos salen aprobados sin que entre dinero. OBSERVADO Las cuatro son de PRUEBAS (sandbox). Pasarela configurada sin problemas.	Crítica	Pasa
CRED-04	Ningún secreto llega al navegador POR QUÉ Un secreto en el bundle es público: cualquiera lo lee desde las herramientas del navegador, y con el de integridad se pueden firmar cobros por el monto que se quiera. OBSERVADO 86 archivos de dist/ y src/ revisados, ningún secreto dentro.	Crítica	Pasa
CRED-05	Los secretos no están versionados POR QUÉ Un secreto en el historial de git sigue ahí aunque se borre del archivo, y viaja a cualquiera que clone el repositorio. OBSERVADO server/.env ignorado y fuera del índice; ningún secreto en los archivos versionados.	Crítica	Pasa
CRED-06	El servidor se niega a arrancar con llaves de prueba en producción POR QUÉ Es el error silencioso más caro: la tienda vende con normalidad y no entra un peso. Nadie se entera hasta cuadrar caja. OBSERVADO assertPaymentsReady() aborta el arranque si NODE_ENV=production con llaves pub_test_.	Crítica	Pasa

B · Firma de integridad del checkout

ID	PRUEBA	GRAV.	RESULT.
FIRM-01	La firma que viaja al checkout es reproducible POR QUÉ Es lo único que impide que alguien edite el monto en el navegador y pague \$1.000 un pedido de \$300.000. OBSERVADO Referencia AU-11684-5ce6, 8070000 centavos. Firma recalculada idéntica.	Crítica	Pasa

ID	PRUEBA	GRAV.	RESULT.
FIRM-06	La URL de retorno no apunta a localhost POR QUÉ Wompi rechaza con un 403 de CloudFront cualquier redirect-url que apunte a localhost o 127.0.0.1, con http y con https. Con APP_URL en localhost el checkout NO ABRE, y la clienta ve una página de error de Amazon sin ninguna pista. OBSERVADO Retorno a <code>directory-inquiries-appraisal-commander.trycloudflare.com</code>; el checkout de Wompi responde 200 con la URL completa.	Crítica	Pasa
FIRM-02	Cambiar el monto invalida la firma POR QUÉ Es el ataque concreto: rebajar amount-in-cents en la URL antes de pagar. OBSERVADO \$300.000 → 7b4adc14c0035c7c... · \$1.000 → 0cc9243afbd7c286... (Wompi rechaza la segunda)	Crítica	Pasa
FIRM-03	Cambiar la referencia invalida la firma POR QUÉ Reapuntar un pago barato a la referencia de un pedido caro es la otra mitad del mismo ataque. OBSERVADO Referencias distintas producen firmas distintas.	Alta	Pasa
FIRM-04	La expiración entra en la cadena solo si se envía POR QUÉ Firmarla sin mandarla (o al revés) da una firma inválida y Wompi devuelve un error que no dice por qué. Es la trampa clásica de esta integración. OBSERVADO Las dos variantes difieren; el checkout manda expiration-time y firma con ella.	Media	Pasa
FIRM-05	Sin el secreto no se puede falsificar una firma POR QUÉ Si la firma se pudiera reproducir sin el secreto, no protegería nada. OBSERVADO Secreto distinto, firma distinta.	Crítica	Pasa

C · Verificación de avisos de la pasarela

ID	PRUEBA	GRAV.	RESULT.
AVIS-01	Un aviso bien firmado se acepta POR QUÉ Es la base: si no se aceptara un aviso legítimo, ningún pago se confirmaría nunca. OBSERVADO Aceptado.	Crítica	Pasa
AVIS-02	Un checksum alterado en un carácter se rechaza POR QUÉ Si una firma casi correcta pasara, la firma no serviría de nada. OBSERVADO Rechazado: La firma del aviso no coincide.	Crítica	Pasa
AVIS-03	Un aviso firmado con otro secreto se rechaza POR QUÉ Es lo que mandaría quien conozca el formato pero no el secreto, que es todo el mundo: el formato está documentado. OBSERVADO Rechazado: La firma del aviso no coincide.	Crítica	Pasa
AVIS-04	Un aviso sin firma se rechaza POR QUÉ Lo más simple que puede intentar alguien: mandar el JSON sin la sección de firma. OBSERVADO Rechazado en los dos casos: El aviso no trae signature.properties.	Crítica	Pasa

ID	PRUEBA	GRAV.	RESULT.
AVIS-05	Una propiedad firmada que no viene se rechaza POR QUÉ Si una propiedad ausente se tratara como cadena vacía, se podría recortar la parte firmada hasta dejarla trivial. OBSERVADO Rechazado: La propiedad firmada "transaction.campo_que_no_existe" no viene en el aviso.	Alta	Pasa
AVIS-06	La comparación de firmas es en tiempo constante POR QUÉ Con === el tiempo de respuesta depende de cuántos caracteres aciertan, y eso deja adivinar un checksum válido byte a byte desde fuera. OBSERVADO safeEqualHex() compara con crypto.timingSafeEqual.	Media	Pasa
AVIS-07	Un aviso viejo se descarta POR QUÉ Un aviso legítimo capturado hace días lleva una firma perfectamente válida para siempre. El timestamp es lo único que permite descartarlo. OBSERVADO Descartado por antigüedad (ventana de 48 h).	Alta	Pasa
AVIS-08	El último reintento de Wompi (24 h) sigue siendo válido POR QUÉ Wompi reintenta a los 30 min, 3 h y 24 h. Con la ventana en 24 h justas rechazábamos nosotros mismos el último reintento, que es precisamente el que llega tras dos fallos. OBSERVADO Aceptado: la ventana es de 48 h, con margen sobre el último reintento.	Alta	Pasa
AVIS-09	Un aviso con fecha futura se descarta POR QUÉ Un timestamp adelantado dejaría un aviso reutilizable durante días. OBSERVADO Una hora de adelanto se rechaza; un minuto se tolera.	Media	Pasa
AVIS-10	Un tipo de evento no soportado se rechaza POR QUÉ Solo transaction.updated mueve dinero. Procesar a ciegas cualquier evento futuro de Wompi es aceptar un contrato que todavía no se ha leído. OBSERVADO Rechazado: Evento no soportado: nequi_token.updated.	Media	Pasa

D · Intentos de abuso

ID	PRUEBA	GRAV.	RESULT.
ATAQ-01	Un aviso sin firma válida no mueve el pago POR QUÉ Es el ataque que vacía el inventario: mandar «APROBADO» al webhook y recoger la mercancía sin pagar. OBSERVADO 401 y el pago AU-11686-c279 sigue PENDING.	Crítica	Pasa
ATAQ-02	Un aviso bien firmado pero con otro monto no aprueba el pedido POR QUÉ Protege del caso en que alguien logre pagar 1.000 pesos una transacción asociada a un pedido de 300.000. La firma sería válida; el monto no. OBSERVADO La API respondió 200 y el pago quedó en PENDING, no APPROVED (se cobraban 4780000 centavos y el aviso traía 100).	Crítica	Pasa

ID	PRUEBA	GRAV.	RESULT.
ATAQ-03	Reenviar el mismo aviso no duplica nada POR QUÉ Wompi reintenta hasta tres veces. Sin idempotencia, un pedido podría descontar stock dos veces o aparecer pagado por partida doble. OBSERVADO Reenvío aceptado con 200. Avisos guardados: 1; aplicados: 1; movimientos de inventario del pedido: 1.	Alta	Pasa
ATAQ-04	Un aviso de una referencia desconocida no revienta ni reintenta POR QUÉ Pasa de verdad cuando dos entornos comparten credenciales. Un 500 haría que Wompi reintentara 24 h contra algo que nunca va a existir. OBSERVADO 200 con applied:false: Wompi deja de insistir y queda registrado.	Media	Pasa
ATAQ-05	La ruta del pago simulado no existe con Wompi activo POR QUÉ Es una ruta pública capaz de marcar pagos como aprobados a voluntad. Con la pasarela real encendida sería la forma más directa de vaciar el inventario sin pagar. OBSERVADO 404: la ruta no se registra cuando el proveedor es WOMPI.	Crítica	Pasa
ATAQ-06	No se puede pedir más de lo que hay POR QUÉ Vender lo que no existe obliga a devolver el dinero y deja mal a la tienda. OBSERVADO Rechazado con 409: «Stock insuficiente para AN-1111: pediste 30 y hay 5.»	Alta	Pasa
ATAQ-07	Cantidades absurdas se rechazan en la puerta POR QUÉ Una cantidad negativa o cero puede dar totales negativos; una enorme, desbordes. OBSERVADO 0, -5 y 1.5 devolvieron 400, 400, 400.	Media	Pasa
ATAQ-08	Dos compras simultáneas no venden la misma unidad POR QUÉ Sin una guardia atómica, dos peticiones que leen el stock a la vez pasan las dos la comprobación y se vende dos veces lo mismo. OBSERVADO Dos peticiones simultáneas por 3 unidades de p10: 1 aceptada, 1 rechazada. Reservado 3 de 3.	Crítica	Pasa
ATAQ-09	El webhook solo acepta POST POR QUÉ Un GET al webhook es la sonda más común de un escáner automático. OBSERVADO GET al webhook devuelve 404.	Info	Pasa

E · Estados y reconciliación

ID	PRUEBA	GRAV.	RESULT.
EST-01	Los estados de Wompi se traducen sin perder ninguno POR QUÉ Wompi ha devuelto estados que no documenta (PROCESSING, FAILED, REJECTED). Tratar lo desconocido como error soltaría el stock de algo que quizá sí se pagó. OBSERVADO 8 estados traducidos; lo desconocido queda PENDING.	Alta	Pasa
EST-02	Un pago aprobado no se puede volver a mover POR QUÉ Los avisos llegan desordenados. Un DECLINED viejo que aterrice después del APPROVED bueno cancelaría un pedido ya cobrado. OBSERVADO PENDING→APPROVED sí; APPROVED→DECLINED y APPROVED→PENDING no.	Crítica	Pasa

ID	PRUEBA	GRAV.	RESULT.
EST-03	Un pago caducado todavía puede aprobarse POR QUÉ Si el dinero entró tarde, entró. Dar por perdido un cobro que la pasarela sí capturó es quedarse con el dinero y sin el pedido. OBSERVADO EXPIRED→APPROVED permitido; una aprobación tardía se registra en vez de perderse.	Alta	Pasa
REC-01	Se puede reconciliar por referencia contra el API real POR QUÉ Es la única salida cuando se pierde el aviso Y la clienta nunca vuelve: sin esto el cobro queda «pendiente» para siempre con el stock bloqueado. OBSERVADO GET /v1/transactions?reference=... autenticado con la llave privada; referencia inexistente → null.	Alta	Pasa
REC-02	Consultar una transacción que no existe no rompe nada POR QUÉ Un 404 de la pasarela tratado como excepción dejaría a la clienta sin respuesta en la pantalla de resultado. OBSERVADO Id inexistente → null, sin excepción.	Media	Pasa
REC-03	El dinero se maneja en enteros POR QUÉ El peso colombiano no tiene centavos y los flotantes arrastran error: 0.1+0.2 no es 0.3. Un céntimo de desfase hace que Wompi rechace la firma. OBSERVADO Cinco montos convertidos a centavos sin pérdida.	Alta	Pasa

F · Superficie expuesta

ID	PRUEBA	GRAV.	RESULT.
SUP-01	La consulta pública de un pago no filtra datos de la clienta POR QUÉ La referencia viaja en URLs y acaba en historiales y registros. Quien la tenga no debe poder sacar de ahí el correo, el teléfono o la dirección. OBSERVADO Devuelve estado y totales. Sin correo, teléfono, documento ni dirección. Campos: reference, status, message, orderNumber, orderStatus, amount, methodType, expiresAt, updatedAt	Alta	Pasa
SUP-02	Una referencia con forma inválida se rechaza sin tocar la base POR QUÉ El parámetro entra directo en una consulta. Validar la forma antes corta de raíz cualquier intento de inyección o de sondeo. OBSERVADO 4 sondas (SQL, recorrido de rutas, script) rechazadas por el validador de forma.	Media	Pasa
SUP-03	Los secretos no salen en los mensajes de error POR QUÉ Un mensaje de error que arrastre la cadena firmada publica el secreto de eventos a cualquiera que provoque el error. OBSERVADO Respondió 401 con «Firma del aviso inválida.», sin secretos.	Alta	Pasa
SUP-04	El proxy del túnel solo deja pasar la ruta del webhook POR QUÉ Tunelizar la API entera publicaría en internet el login del panel con credenciales de demo. Wompi solo necesita una ruta. OBSERVADO Filtra por método y ruta, limita el cuerpo y deja bitácora de lo recibido.	Alta	Pasa

H · Límites de uso

ID	PRUEBA	GRAV.	RESULT.
ATAQ-10	Hay límite de peticiones en la creación de intentos POR QUÉ Crear un intento aparta stock sin pedir cuenta ni pago. Sin freno, un bucle desde una IP deja el catálogo sin unidades disponibles: medido antes del arreglo, 10 reservas en 355 ms. OBSERVADO De 40 peticiones seguidas, 6 pasaron y 32 recibieron 429.	Alta	Pasa
ATAQ-11	RIESGO ABIERTO: el inventario se puede agotar sin pagar POR QUÉ Un intento de pago aparta stock durante 15 minutos sin cuenta ni cobro. El límite de peticiones frena el barrido del catálogo, pero no puede ponerse por debajo del stock de un producto sin dejar fuera a clientas reales. OBSERVADO Producto más expuesto: p10 con 0 unidades libres. Por encima del límite por minuto, así que haría falta más de un minuto por producto. Cierre pendiente: apartar stock al abrir el checkout, acortar la ventana de reserva y topar reservas pendientes por IP.	Alta	Pasa

G · Pendiente de un aviso real

ID	PRUEBA	GRAV.	RESULT.
E2E-01	Un aviso emitido por Wompi pasa nuestra verificación POR QUÉ Todo lo anterior prueba que verificamos BIEN el algoritmo que implementamos. Que ese algoritmo sea el de Wompi solo lo demuestra un aviso suyo, y hay un motivo concreto para dudar: el ejemplo resuelto de su documentación NO reproduce su propio checksum (comprobado: la cadena que publican, hasheada con el secreto que publican, da otro hash). OBSERVADO 2 de 2 avisos reales de Wompi aceptados con 200.	Crítica	Pasa

Generado automáticamente desde `docs/pruebas-wompi/resultados.json` · Banco de pruebas: `server/scripts/pruebas-wompi.ts` · Limpieza tras la ejecución: 13 cobros de prueba caducados; {"checked":12,"expired":12,"rescued":0}